NTI • PROJECT RECOGNITION • DIGITAL DESIGN USING FPGA

FPGA-Based Advanced Automotive ECU Suite

Deterministic hardware ECUs for vehicle safety and performance

Designed, simulated and verified in Verilog

UNDER THE SUPERVISION OF

Eng. Mohammed Salah Aboshosha
Eng. Omar Mostafa Aladawy

PROJECT TEAM

Amr Waheed Mohammed
Aly Hamdy Aly
Abdelrahman Youssef
Zeyad Khaled Abd Elhamid
Adham Amr Mohammed

October 2026 • Verilog / SystemVerilog • Questasim • Vivado • Icarus Verilog



ABS + Traction
Slip > 15 → torque cut



Smart RPM Limiter
9k soft / 10k hard cut



Active Spoiler
80 / 150 km/h + air-brake



Obstacle Detection
Always-on • 3 zones + buzzer



Drive Modes
Eco • Normal • Sport • Track

MOTIVATION

Where did the idea come from?

02

PASSION + OBSERVATION

We love car mechanics — vehicle dynamics is our passion.

A modern car runs 50–100 ECUs — A/C to brakes.

Economy ECUs lag — shared sequential CPUs, felt in response.

The spark: what if EVERY ECU was dedicated hardware?

FROM IDEA TO PROJECT

Physics → Verilog → FPGA
car behavior into synthesizable RTL

Picked 5 real problems: Braking, Revs , Aero, Parking, Feel.

One ECU per problem — sense, decide, act in hardware.

Goal: prove it in simulation, ready for silicon.

A CAR IS A NETWORK



Every glowing node = one ECU

50–100 ECUs
in every modern car

PROBLEM

Sequential MCUs cannot keep up

03

THE BOTTLENECK

```
CPU: task 1 → task 2 → …
one at a time + interrupts + OS jitter
```

Shared CPU runs everything in turn.

Latency + jitter — response time varies.

OK for comfort (A/C, dashboard) — risky for safety.

WHAT BREAKS

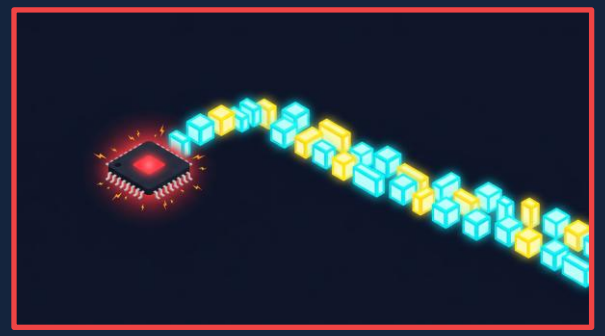
Late ABS reaction → wheel locks, longer stop.

Missed rev-limit → engine damage over time.

Sluggish throttle → dead or jumpy pedal feel.

One crash hangs all — single point of failure.

ONE CPU, ENDLESS QUEUE



Sequential tasks pile up

1 CPU → N tasks
queuing delay + jitter!

SOLUTION

One dedicated hardware ECU per job

04

SOLVED IN HARDWARE

```
BRAKE || SPOILER || RPM
every ECU at the same instant
```

Verilog HDL builds real circuits, not software.

True parallelism — 5 ECUs, zero waiting.

Deterministic — same input, same cycle, same output.

DESIGN RULES

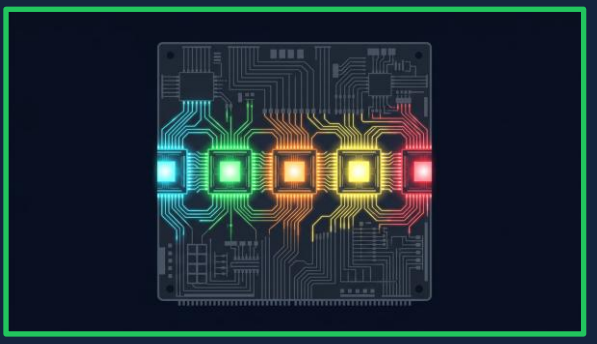
Synchronous RTL — one clock, clean resets.

Modular ECUs — each senses, decides, acts.

Synthesizable — no simulation-only tricks.

Self-tested — a testbench proves each ECU.

5 ECUs, ONE CLOCK



All blocks fire together

5 DEDICATED CIRCUITS



TOP MODULE

One suite. Five hardware ECUs.

05



Obstacle Detect

IN: echo pulses (always-on)

▼

OUT: 3 LEDs + buzzer

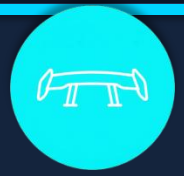


Drive Modes

IN: throttle, mode

▼

OUT: motor PWM



Active Spoiler

IN: speed, brake

▼

OUT: servo PWM, air-brake LED



ABS + Traction

IN: wheel speeds, brake, PWM

▼

OUT: cut PWM, ABS pulse, LEDs



RPM Limiter

IN: RPM, speed, brake, throttle

▼

OUT: spark retard, fuel cut

Top level: Sensors → FPGA fabric (parallel RTL) → Actuators • Shared: speed • brake • throttle • PWM

Next: each ECU in 90 seconds — idea, inputs/outputs, and its test

Obstacle Detection System

IDEA (1-2-3)

1. **Always-on ultrasonic** echo timing (not just reverse).
2. **Counter = distance** → Safe/Mid/Close/Danger zones.
3. **Alerts escalate**: LEDs + buzzer slow → continuous.

```
IN:  echo_pulse (ultrasonic sensor)
OUT: led_green/yellow/red, buzzer_out
```

ECHO TIMING



Counter starts → echo returns → stops

LED BAR + SENSORS



Real hardware: LED bar + 4 sensors

TEST • tb_parking_system
short/long echo pulses → correct zone, LEDs and beep rate each time

Drive Mode Controller

IDEA (1-2-3)

1. **LUT per mode**: Eco calm → Normal → Sport → Track beast.
2. **Slew limiter** ±1 step — no drivetrain shock.
3. **8-bit PWM** drives the motor smoothly.

```
IN:  throttle_in (2-bit), drive_mode (2-bit)
OUT: motor_pwm
```

REAL MODE DIAL



Eco → Track at a twist

TEST • tb_Drive_Mode

all 4 modes × throttle steps → curves match LUT, ramp climbs smoothly with no jumps

Active Spoiler System

08

IDEA (1-2-3)

1. **Speed stages**: <80 closed, 80–150 cruise, >150 track.
2. **Brake at speed** → full air-brake (255).
3. **Smooth servo**: ±1 steps, no jerks.

```
IN:  vehicle_speed (9-bit), brake_pedal
OUT: spoiler_pwm (servo), airbrake_led
```

TRACK WING + AIR-BRAKE



GT wing: downforce on demand

TEST • tb_Active_Spoiler

speed ramp through all stages + brake stab at speed → angles track targets, air-brake fires

ABS + Traction Control

09

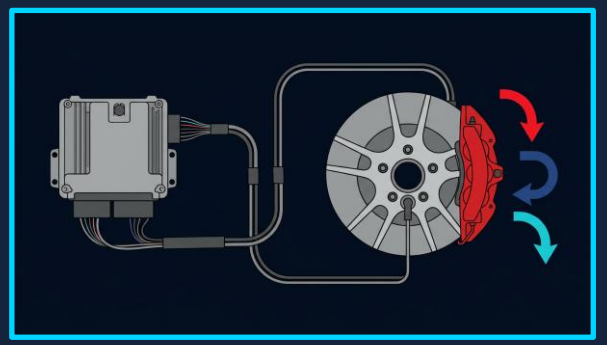
IDEA (1-2-3)

1. **Compares front vs rear speed** every clock.
2. **Braking + slip over 15** → ABS pulse, no lock-up.
3. **Driving + wheel spin** → torque PWM halved (TCS).

```
IN:  front/rear speed (8-bit), brake_pedal, ecu_pwm

OUT: final_pwm, abs_brake_pulse, TCS/ABS LEDs
```

ABS CONTROL LOOP



ECU modulates pressure: increase → hold → decrease

STABILITY: TCS ON



Green line = stable (TCS). Red = sliding.

TEST • tb_Traction_ABS

lock-up braking + launch spin + normal cruise stimulus → pulses, halving and LEDs verified on waveforms

Smart RPM Limiter

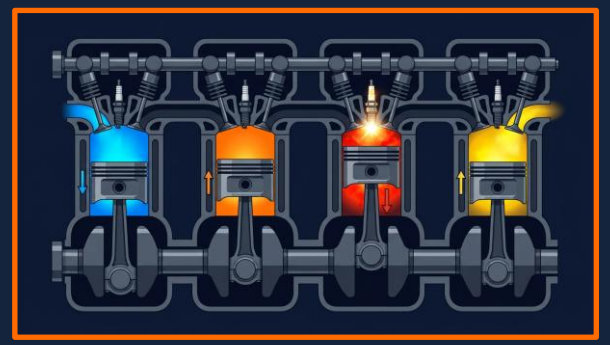
10

IDEA (1-2-3)

1. **4k launch control** holds revs via spark retard.
2. **9k soft cut** retards spark (warning zone).
3. **10k hard cut** kills fuel (danger zone).

```
  IN:  engine_rpm, vehicle_speed, brake_pedal,
throttle_in
  OUT: spark_retard, fuel_cut
```

4-STROKE CYCLE



Spark fires the power stroke

REDLINE ZONES



Needle must never cross the redline

TEST • tb_Smart_RPM

RPM sweep 0→10k + launch combo (speed 0 + brake + full throttle) → cuts fire at exact thresholds

DEMO

Running the project live

HOW WE RUN IT

```
$ iverilog -o sim tb/tb_X.v rtl/X.v
$ vvp sim   →   GTKWave sim.vcd
```

Any module runs standalone in seconds.

Same flow on Questasim / Vivado.

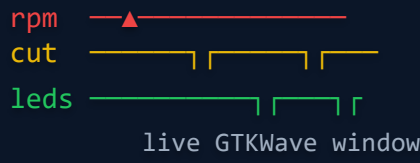
Backup: recorded waveforms if live fails.

WHAT YOU WILL SEE

RPM sweep 0→10k: cuts fire at 4k / 9k / 10k.

Obstacle echoes: zones + beep rate change live.

ABS slip: torque halves the instant slip trips.



RESULTS

After the fix: measured outcomes

12

BEFORE (MCU)

Sequential tasks — systems wait in line.

Jittered response — varies run to run.

One CPU — single point of failure.

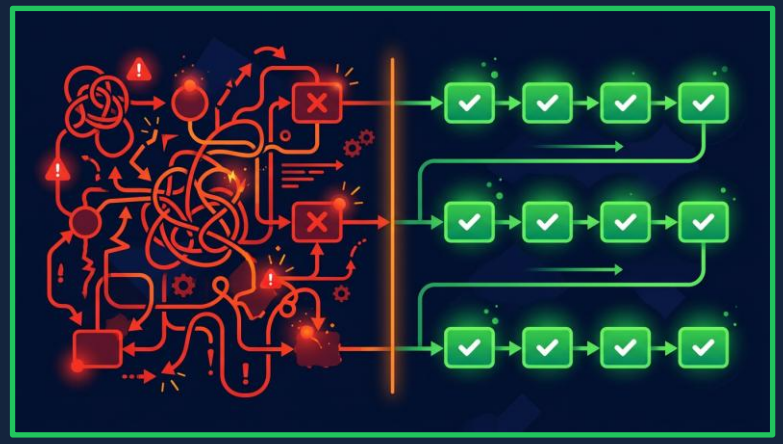
AFTER (FPGA ECUs)

All 5 parallel — react same clock cycle.

Deterministic — identical every single run.

Isolated ECUs — one fault cannot hang others.

CHAOS → ORDER



Same workload. Hardware discipline.

5/5 testbenches pass • zero latches, synchronous resets • 100% synthesizable RTL • documented + on GitHub

Why FPGA — and the ASIC path

PARALLELISM

5 real circuits at once — no scheduler, no waiting.

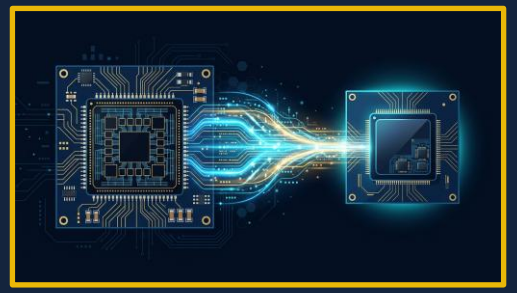
DETERMINISM

Cycle-exact timing — provable, testable, certifiable.

RELIABILITY

No OS, no crashes — plus power-efficient logic.

FPGA → ASIC



Prototype now, tape out at volume

Low volume → FPGA wins • High volume → same RTL tapes out to ASIC • Our code is ready for both

Who uses it — and why it matters

TARGET USERS

Motorsport teams (GT / Formula) — every ms is lap time.

Supercar makers — low volume, extreme demands.

EV startups — fast iteration on FPGA platforms.

Track aftermarket — retrofit kits for enthusiasts.

IMPACT

Safety: shorter stops, protected engines.

Performance: sharper response, faster laps.

Platform: proven RTL blocks reusable in ADAS/EV.

BUILT FOR THE TRACK



F1 • Supercars • EVs

Every millisecond matters — that is the business case

NEXT STEPS

Roadmap to the track

1 — TOP-LEVEL ECU

Integrate all 5 + arbiter logic.

2 — FPGA BRING-UP

Artix-7 / Zynq + timing constraints.

3 — CAN-BUS LINK

Talk to the real car network.

4 — HIL TESTING

Real sensors + actuators on bench.

ROADMAP TO THE TRACK



4 milestones. One destination.

Thank you!

Questions & Discussion

github.com/alyhamdy879/automotive-ecu-fpga-main

RTL • Testbenches • Technical report

Amr Waheed Mohammed • Aly Hamdy Aly • Abdelrahman Youssef • Zeyad Khaled Abd Elhamid • Adham Amr Mohammed

NTI • October 2026